

Universidade Estadual de Maringá - Departamento de Informática
Programação de Sistemas
Prof. Marcelo Malheiros

LISTA SOBRE PROGRAMAÇÃO COM PROCEDIMENTOS E FUNÇÕES EM C

Cada exercício deve ser implementado em um programa separado. Sugere-se o padrão **exer_0.c**, contendo uma função **main()** que chama os procedimentos e funções especificados, desta forma implementando os testes solicitados.

Exercício 0: Escreva um programa que define um procedimento chamado **media**, que recebe dois valores inteiros como parâmetro, então calculando e imprimindo a média aritmética destes na saída. Teste chamando **media(27, 13)**.

Resultado esperado:

20

Exercício 1: Escreva um programa que define um procedimento chamado **impares**, que recebe um inteiro **n** como parâmetro, então imprimindo em uma mesma linha os **n** primeiros números ímpares começando em 1. Teste chamando **impares(10)**.

Resultado esperado:

1 3 5 7 9 11 13 15 17 19

Exercício 2: Escreva um programa que define um procedimento chamado **matriz**, que recebe dois inteiros **m** e **n**, indicando o número de linhas e colunas, respectivamente. Este procedimento deve imprimir na tela uma saída indicando os índices de cada elemento de uma matriz com tais dimensões. Teste com os parâmetros para uma matriz de 3 linhas e 5 colunas.

Resultado esperado:

0,0 0,1 0,2 0,3 0,4
1,0 1,1 1,2 1,3 1,4
2,0 2,1 2,2 2,3 2,4

Exercício 3: Escreva um programa que define um procedimento chamado **triangulo**, que recebe um inteiro **n** e imprime na tela um triângulo feito de dígitos. Teste com o valor 5.

Resultado esperado:

1
12
123
1234
12345

Exercício 4: Escreva um programa que define um procedimento chamado **fatoriais**, que recebe um parâmetro inteiro **n** e imprime na tela a expansão e os resultados das fatoriais de 1 até **n**. Teste com o valor 5. Dica: na expansão o primeiro termo é 1, e os demais sempre são precedidos de " x ".

Resultado esperado:

```
1! = 1 = 1
2! = 1 x 2 = 2
3! = 1 x 2 x 3 = 6
4! = 1 x 2 x 3 x 4 = 24
5! = 1 x 2 x 3 x 4 x 5 = 120
```

Exercício 5: Escreva um programa que define a função **distancia**, que recebe como parâmetro dois pares de valores reais (**x1** e **y1**, **x2** e **y2**), representando pontos em um espaço de duas dimensões. Esta função deve calcular a distância euclidiana entre estes dois pontos e retornar o valor real calculado. Teste com os pontos (4.0, 6.0) e (7.0, 2.0). Dica: use a função **sqrt()** disponível no *include* **<math.h>**.

Resultado esperado:

5.0

Exercício 6: Escreva um programa que define a função **fatorial**, que recebe como parâmetro um valor inteiro positivo **n** e devolve o valor real **n!** como valor de retorno. Teste com o valor 5.

Resultado esperado:

120.0

Exercício 7: Escreva um programa que define a função **serie**, que recebe como parâmetro um valor inteiro positivo **m** e devolve o valor da série $1! / 1 + 2! / 2 + 3! / 3 + \dots + m! / m$. Copie a função **fatorial** definida na prática anterior, chamando esta como parte de calcular o somatório. Teste com o valor 5.

Resultado esperado:

34.0

Exercício 8: Escreva um programa que define a função **digitos**, que recebe como parâmetro um valor inteiro positivo **a** e devolve o número de dígitos que **a** possui. Dica: o resto da divisão (ou módulo) de 456 por 10 é 6, enquanto que o quociente da divisão (inteira) é 45. Teste com os valores 8, 27 e 456.

Resultado esperado:

```
1
2
3
```

Exercício 9: Escreva um programa que define a função **narcisista**, que recebe como parâmetro um valor inteiro positivo **b** e devolve um valor lógico (inteiro: igual a 0 se falso, diferente de 0 se verdadeiro) indicando se **b** é um número narcisista. Um número narcisista é aquele que é igual à soma de seus dígitos elevados a uma potência igual ao número de dígitos. Por exemplo, o número 153: elevando cada um dos seus dígitos ao cubo (pois 3 é o número de dígitos) e somando os resultados, temos $1^3 + 5^3 + 3^3 = 153$. Dica 1: copie a função **digitos** definida anteriormente e chame ela de dentro de **narcisista**. Dica 2: calcule a potência por multiplicações repetidas, como **x*x** ou **y*y*y**. A função **main** deve então testar todos os números inteiros entre 100 e 10000, imprimindo na tela em uma linha os que são narcisistas.

Resultado esperado:

153 370 371 407 1634 8208 9474